Increasing Network Security with eBPF

Assignment #1 - State of the art

Faculty of Sciences · University of Porto

António Lanção (up202205416)
David Sá (up202303580)
Henrique Teixeira (up202306640)

Network Security 2026/27
01 November 2026

Abstract. Extended Berkeley Packet Filter (eBPF) lets verified programs run at hook points spanning the whole kernel, from packet reception (XDP) to permission checks (BPF LSM), with maps as shared state. This makes it a substrate for security systems rather than a security product, and two questions follow. First, which hook should a defense use: we compare hook families on packet state, verdicts and cost, and derive two use cases from them. Second, is the substrate itself trustworthy: we survey the execution and safety model, the verifier, CO-RE portability, and the privilege boundary, and build a threat model for eBPF deployments from published cross-container attacks and verifier critiques. Defense systems are then compared as architectures: production (Cilium, Tetragon) against research (bpfbox, BPFContain), finding that policy authorship, not capability, separates them. We close with a prototype proposal for Assignment 2, a continuous process profiler feeding a rule-based malware detector, with evaluation criteria tying detection fidelity and overhead to the least-privilege deployment the threat model demands.

1 Introduction

Classic kernel extension mechanisms force a choice: loadable kernel modules run arbitrary code with full kernel privilege and must match the running kernel exactly, while fixed in-kernel subsystems such as netfilter or the LSM framework only implement the policies their authors anticipated. Extended Berkeley Packet Filter (eBPF) removes the choice by letting untrusted code run **inside** the kernel safely: programs are compiled to a small instruction set, statically proven safe by an in-kernel verifier before they ever run, JIT-compiled to native code, and attached to hook points that span the whole range of kernel work kernel work, from the earliest moment a packet exists from the earliest moment a packet exists (XDP, in the driver, before any allocation) to the kernel's own permission checks (BPF LSM), with maps as shared state between kernel programs and user space [1], [2], [3].

For network security this makes eBPF a **substrate** rather than a product: it provides safe programmability at kernel hook points, and defense systems are built on top of it as architectures of programs, maps and user-space control planes. Two things follow, and both structure this report. First, the choice of hook is a design decision with security consequences with security consequences: each position in the kernel trades each position in the kernel trades visibility for cost, so the same attack is defensible at different prices depending on where the defense sits. Second, the substrate is itself security-sensitive: the verifier is a large, hand-written component of the Trusted Computing Base whose bugs are kernel bugs [4], and the systems built on eBPF add their own seams add their own seams; cross-container attacks that confuse the identity mapping or exploit the policy data path show the confinement breaking at the joins, not the machinery [5].

This report compares the state of the art along those lines. Its contributions:

- a description of the eBPF execution and safety model model, the load → verify → JIT → attach pipeline, the verifier's three families of checks, and maps as the only durable memory memory, separating separating what the machinery guarantees from what it does not;
- a hook-by-hook comparison of the security-relevant attachment points (XDP, tc, cgroup/socket, tracepoints/kprobes, BPF LSM), instantiated with two use cases that lead to different hooks from the same question;
- a comparison of production systems (Cilium, Tetragon) with research confinement systems (bpfbox, BPFContain), showing each as a point in the same design space of hooks × programs × maps × control plane;
- a threat model for eBPF itself, assembling the verifier-as-TCB critique and the cross-container attack literature into adversaries and mitigations, which the prototype of Assignment 2 will partially validate.

The remainder follows that order: §2 the execution and safety model, §3 hook points and use cases, §4 security systems, §5 the threat model against eBPF, §6 limitations and open challenges, §7 the proposed prototype and its evaluation criteria, and §8 the implementation roadmap.

2 The eBPF execution and safety model

2.1 Program lifecycle: bytecode, loading and JIT

eBPF programs are compiled from restricted C into bytecode for a small register-based virtual machine: eleven 64-bit registers that map one-to-one onto hardware registers on the supported 64-bit architectures, a fixed instruction set, and a single calling convention shared with the C ABI C ABI, which which is what makes just-in-time translation to native code a faithful, mechanical restatement rather than a second implementation [1], [2].

Loading goes through a single entry point, the `bpf()` system call. The kernel takes the bytecode, hands it to the verifier (Section 2.2), and only after the proof succeeds passes it to the JIT compiler, which translates each instruction to a small block of native machine code with no runtime checks, interpreter, or sandbox left in the hot path. The proven bytecode is the safety artifact; the JIT preserves its properties and invents no behavior of its own. The result is native-speed execution with zero per-event safety overhead: verification is paid once at load time, execution is fast forever.

A program only does something once it is **attached** to a hook point: the kernel invokes it as an event handler whenever the hooked event fires fires: a packet arriving at a driver, a syscall, a security decision. a packet arriving at a driver, a syscall, a security decision. Loaded programs and their maps are reference-counted by the kernel and can be pinned to a filesystem path (bpffs) so they outlive the loader process [6].

2.2 The verifier

eBPF code runs in kernel address space, where a single bad memory access corrupts the machine every process depends on. Since the requirement is certainty about **all** executions executions, including inputs an including inputs an attacker crafts attacker crafts, before before the program ever runs, runtime mechanisms are categorically insufficient: a sandbox needs a floor below the kernel and there is none, and testing only covers the inputs somebody thought of [7]. The verifier therefore **proves** safety by static analysis over all reachable paths, in two passes over a directed-acyclic graph of the program's control flow [2].

The proof tracks an abstract state state: register types, pointer offsets register types, pointer offsets and possible value ranges ranges, through through every path, and enforces three families of checks:

- **Termination.** General termination is undecidable, so the verifier restricts the class of allowed programs instead: historically it required an acyclic control-flow graph (no loops at all), and modern kernels admit bounded loops within an instruction budget. The check is mechanical precisely because the program class is restricted.
- **Memory safety.** Every load or store must be provably in bounds on every path: packet accesses against the `data_end` pointer in the context, map values against the map's declared value size, stack accesses against ranges previously written. Bounds checks in the program (e.g. `if (ptr + offset > data_end)`) are not a courtesy not a courtesy: they are how the programmer hands the verifier the proof, and the verifier refines ranges at every branch.
- **Type safety.** Each register carries a type alongside its range, and every instruction and helper call must be type-compatible: a packet pointer cannot be used as a map key, a map lookup's return must be checked for NULL before dereference. Type confusion is the classic kernel exploitation primitive, and the verifier refuses to let a program create one [2].

The verifier does **not** check that the program does what its author intended, nor that it is efficient; its sole purpose is protecting the kernel's internals [2]. This makes it the trusted computing base (TCB) of every eBPF deployment: the component every other safety property depends on, and which must itself be trusted rather than verified. The analysis is also an over-approximation: at merge points, states are unioned and precision is lost, so correct programs can be rejected rejected, the practical cost of the practical cost of which grows with program complexity, as Jia et al. argue [4]. PREVAIL [8] formalizes the same analysis as an abstract interpretation with a machine-checked soundness argument, precisely to address what the hand-written in-kernel verifier cannot prove about itself.

2.3 Maps, ring buffers and helpers

An eBPF program is an event handler: it is invoked, runs, and returns, with no state that survives between invocations. Since nearly every security task needs memory across events events: a rate limiter needs a rate limiter needs per-source counters, a policy needs its rule set rule set; state must live state must live outside the program, and that object is the **map**: a typed key→value store allocated by the kernel, read and written by programs through helper calls, and equally reachable from user space [1], [2]. Maps thus serve three roles at once: persistent storage between invocations, coordination between programs attached at different hooks, and the kernel↔user-space channel that a control plane reads and updates.

The map type encodes the access pattern. Security-relevant ones include hash maps (key→value lookup, e.g. per-IP counters), arrays (index→value), LPM tries (longest-prefix match, the data structure behind CIDR-based policy), per-CPU variants (each CPU has its own copy, so hot-path updates never take a lock), and ring buffers, which stream events to user space with per-CPU buffers and reserved records. Helpers and kfuncs are the kernel-provided interface a program calls to do anything beyond pure computation: map lookup, packet manipulation, getting the current process and cgroup identity, redirecting a packet. The verifier checks every helper call's argument types at load time, which is what keeps the interface safe despite the kernel backing it.

Maps are also where **policy** lives. A confinement system's LSM programs are generic; which container may open which file is data in a map the trusted user-space agent loads. This split (enforcement logic in the kernel, policy as data) is the pattern every system in Section 4 follows, and also the attack surface He et al. exploit [5].

2.4 Portability: BTF and CO-RE

An eBPF program compiled against one kernel's structures will not load on another: offsets of fields in `struct` sock, or even the presence of a field, differ across kernel versions and configurations. The classic answers , or compile per kernel, or require vendors to keep stable kernels , both break the "policy as data" deployment model of §2.3.

BPF CO-RE ("Compile Once, Run Everywhere") fixes this with a division of labour [1], [9]:

- **BTF** (BPF Type Format) is the compact type description the kernel ships alongside itself: for each structure, which fields exist at which offsets.
- The **compiler** leaves the structure accesses in the object file as **relocations** ("field X of type Y") instead of hard-coded offsets.
- **libbpf**, at load time, matches those relocations against the running kernel's BTF and patches in the right offsets; where a field genuinely does not exist, a well-named compatibility check (`bpf_core_field_exists`) lets the program take a fallback path.

The verifier's guarantees are unaffected : the program is patched **before** verification , so so safety is not traded for portability. For security tooling this is what makes one shipped policy binary run across a fleet of heterogeneous kernels, the deployment assumption both Cilium and the research systems of Section 4 make.

2.5 Verifier safety vs. authorization to load

> **Key distinction required by the brief**
>
> The verifier answers *"is this program safe to run in the kernel?"*; the privilege model answers *"who may load and attach it?"*. A program the verifier accepts can still be malicious.

The privilege model is separate from, and older than, the verifier: it decides who may reach the `bpf()` system call's privileged commands at all.

- **Root as gatekeeper.** Historically, loading eBPF programs required `CAP_SYS_ADMIN` , but the verifier prevents harm to the kernel, but whoever can load programs already holds kernel-wide power [2].

- **The modern split.** Since kernel 5.8 the monolith is divided: loading programs uses `CAP_BPF` (plus `CAP_PERFMON` for kprobes/tracepoints, and `CAP_NET_ADMIN` for network-attached programs) , narrow, purpose-built capabilities instead of "effectively root".
- **Unprivileged BPF.** Classic-BPF filters (socket filters, seccomp) are loadable without privileges; `kernel.unprivileged_bpf_disabled` disables exactly this path, and distributions increasingly ship it off by default [1][2].
- **Delegation without privilege.** A container or service should be able to load **its own** programs without holding host capabilities. The BPF token mechanism (merged for kernel 6.9′s bpffs delegation) enables this: a privileged manager creates a token tied to a bpffs subtree and hands the token's file descriptor to the container, which can then load programs and create maps , scoped to that subtree, not to the host [3][4].

The distinction matters for the threat model of section 5: every "who may load what" question is answered on this layer, not the verifier's.

3 Hook points for security

3.1 Overview of hooks

The same verified bytecode means different things depending on where it is attached: the hook determines what the program can see, what it can enforce, and what each decision costs. XDP runs in the driver **before** the `sk_buff` exists exists, so the program sees the raw frame and the program sees the raw frame and parses headers itself, and its drop is the cheapest decision the kernel can make about a packet, which is why it is the first-line defense against volumetric attacks [2]. tc hooks sit after the skb is built: parsed headers, flow state and marks are already available, so decisions that depend on stack context context (per-flow classification, per-flow classification, rate limiting, egress filtering filtering) belong there. belong there. Tracepoints and kprobes are not on the packet path at all: they attach to kernel execution and observe the arguments of kernel functions (stable pre-chosen events vs. near-arbitrary instructions instructions, stability against stability against reach). cgroup/socket hooks see operations of a workload's sockets. BPF LSM attaches to the kernel's permission checks and is the only hook whose return value is a **verdict**: it is what turns a system from observation into enforcement [3]. The rule for choosing is derived from position, not speed: **attach where the information you need comes into existence.**

Table 1. Security-relevant eBPF hooks.

Hook	Context available	Enforcement actions	Overhead	Position
XDP	Raw frame, pre-skb	Drop, pass, TX, redirect	Cheapest; line rate	driver RX
tc	Post-skb, headers parsed	Drop, accept, redirect, classify	skb cost already paid	ingress/egress
cgroup/socket	Workload, socket	Deny or restrict socket ops	Per-socket, no skb cost	socket layer
kprobe/tracepoint	Kernel function args	Observe only	Low	anywhere
BPF LSM	Security check, credentials	Allow / deny	Return verdict on the LSM check; function-entry cost	LSM hooks

3.2 Use case A: volumetric / DDoS filtering

A server under a volumetric attack receives millions of junk packets per second, identifiable as garbage from header bytes alone. Header bytes are present in the raw frame, so identification does not require stack context context, and and under attack the weapon is **cost per packet**, so the deciding factor is which drop is cheapest. XDP is the answer: the program parses IP/TCP headers itself and drops the flood before the kernel allocates an `sk_buff`, so the attack's packets never cost more than a few instructions each; the XDP paper's DoS experiment shows mitigation at line rate while the stack is saturated [2]. State the filter needs between packets (per-source counters, rate windows) lives in per-CPU maps, so the hot path never contends. tc **can** filter the same bytes, but each drop happens after the skb is already paid for paid for: the wrong economics for a flood, the right ones the wrong economics for a flood, the right ones once per-flow state or egress policy matters. The conventional baseline, nftables, sits in the stack after skb allocation; comparing the two is Assignment 2′s throughput measurement.

3.3 Use case B: container-aware connection control

Consider the same server, now running untrusted workloads in containers, with a policy requirement: **only the image-fetch service may open connections to the registry network; no container may bind the host's service ports**. This question is not about packets but about **who is allowed to initiate what** , and and the identity "which container initiated this socket operation" does not exist at XDP's layer at all. The information comes into existence where the socket operation is performed, so the

program attaches there: cgroup/socket hooks see the workload's `connect(2)`, `bind(2)` and friends with the cgroup context attached, and return a verdict that restricts that workload's sockets , enforcement without parsing a single packet. State (which container may reach which network) lives in maps updated by the trusted user-space agent, shared with the enforcement program.

This mirrors use case A's shape (identify, decide in-kernel, keep state in maps) but but the **question** chose a different hook, one layer away from the network. It is also BPFContain's starting point, one hook family further out: attach to the LSM checks that guard socket operations, and the same policy now covers containers uniformly, with credentials in the context rather than in the socket [3], [10]. Both use cases together define the scope for Assignment 2: XDP answers "what may this packet do", the socket/LSM family answers "what may this workload do", and a secure system needs the answer to both.

4 eBPF security systems

4.1 Production: Cilium and Tetragon

Cilium applies the eBPF substrate to **networking policy**: instead of IP addresses and port rules, it identifies workloads by labels carried into the kernel as identity, and enforces policy at the L3/L4 and HTTP layers [6]. Enforcement attaches where the traffic already flows , XDP or tc at ingress/egress, and socket-level programs for per-workload rules , so so the same identity model drives routing, load balancing and policy in one place.

Tetragon, from the same ecosystem, targets **process and kernel behaviour** rather than the network: tracing policies attach kprobes and tracepoints to security-relevant kernel functions, correlate events with the process, credentials and container that caused them, and can enforce by denying the call before it completes [11]. Because a policy is data (YAML saying "watch these functions, filter on these arguments"), the in-kernel logic stays generic while the site-specific rules live in user space, the same split as §2.3′s maps.

4.2 Research: bpfbox and BPFContain

The production systems above ship policy for their own orchestration model. The research line asks whether eBPF can **replace** the patchwork of confinement mechanisms (namespaces, cgroups, seccomp, AppArmor/SELinux) with one coherent mechanism.

bpfbox is the proof of concept: per-process confinement policies that can restrict at the **userspace function**, **system call**, **LSM hook** and **kernelspace function** boundaries , granularity no existing mechanism offers, in under 2000 lines of in-kernel code [12]. A policy can require, for example, that `execve` calls originate in a specific function of the confined binary, not just that certain binaries may be executed. In its web-server benchmark, alerting on every security operation costs about 8.4% throughput , slightly less than AppArmor in a comparable configuration , showing the generality is affordable.

BPFContain, by the same authors, generalizes bpfbox from processes to **containers**: a YAML policy language with default-deny and default-allow modes, semantically meaningful permissions on top of the raw LSM API, and integration with existing container managers without modifying their source [10]. Its diagnosis is the inverse of Tetragon's opportunity: the kernel has no unified container abstraction, so isolation is today a mishmash of mechanisms whose combined policies are hard to audit , and and a failure in any one exposes the container. eBPF's LSM hooks close that semantic gap at runtime, with no kernel patches.

4.3 Comparison

Table 2. Production vs. research eBPF security systems.

System	Policy level	Hooks used	Identity	Reported overhead
Cilium	Labels → identity; L3/L4 + HTTP rules	XDP, tc, socket	Workload labels shared kernel-wide	None published; claims in-kernel filtering keeps agent cost off the data plane
Tetragon	YAML tracing/ enforcement policies	kprobes, tracepoints	Process / container from events	None published; in-kernel filtering bounds per-event cost
bpfbox	Confinement policy, per-process	LSM hooks + kprobes	Process under confinement	≈8.4% web-server throughput, under 2000 kLOC
BPFContain	YAML, container-level, default-deny/ allow	LSM hooks + critical functions	Container	Under 16% across Phoronix benchmarks, on par with AppArmor

Overhead is directly comparable only in the research row: bpfbox and BPFContain publish benchmark methodologies and numbers (web-server throughput, Phoronix suites vs. AppArmor), while neither Cilium nor Tetragon publishes comparable figures, reporting only qualitative claims of low overhead from in-kernel filtering. This asymmetry is itself a finding, one the prototype of Assignment 2 will treat as a baseline to produce.

The two lines answer different questions. Production systems assume the orchestration model and buy into it , identity from labels, events from the runtime , trading generality for operational simplicity. Research systems start from the threat model: what a confined process should be able to do, expressed directly against kernel boundaries, at the cost of writing and auditing policy without a manager's help. The substrate is the same in both; what differs is who defines identity and how far below the API the policy reaches.

5 Threat model for eBPF itself

5.1 Assets, trust boundaries and assumptions

The assets are kernel integrity (memory, control flow, module state), the isolation between containers sharing a kernel, the integrity of policy and telemetry consumed by the systems of Section 4, and secrets that live in kernel memory or in other containers' address spaces.

The trust boundaries cross twice. In the loading path, untrusted bytecode crosses from user space into the kernel at the `bpf()` syscall: the verifier and JIT are the gate, and everything they let through is trusted at runtime [4]. In the shared-kernel path there is no boundary at all for tracing programs: a program attached inside container A can read processes and memory of the host and of container B, because eBPF tracing is not restricted by UID or PID namespaces [5].

We assume the standard container protections are enabled (Seccomp, AppArmor/SELinux, KASLR, SMEP/SMAP, non-executable memory) and that the kernel itself has no exploitable memory-safety bug, following the threat model of [5]: the attack surface we model is eBPF's normal features, not a kernel exploit. We also assume the systems of Section 4 are deployed in good faith; a compromised or malicious agent is out of scope.

5.2 Adversaries and attacks

- **Malicious or compromised loader.** Whoever controls the loader can decide which programs are attached, but cannot make the verifier accept unsafe bytecode. The real risk is weaker: a loader with ambient `CAP_SYS_ADMIN`/`CAP_BPF` privilege that loads unsigned programs is a supply-chain choke point, since program provenance is not checked [4].
- **Over-privileged container.** A container granted eBPF capabilities can use tracing features (kprobes, raw tracepoints) to read any process's memory and opened files across containers (`bpf_probe_read_user`) and to kill any process (`bpf_send_signal`) with no kernel exploit at all [5]. The attacker needs one of `CAP_SYS_ADMIN`, or `CAP_BPF` plus `CAP_PERFMON`; over 2.5% of Docker Hub images He et al. measured already have the required permissions.
- **Unsafe helper or program type.** Helpers are escape hatches around the verifier's restrictions, and they also escape its guarantees: a fully-verified program that calls `bpf_probe_write_user` writes arbitrary process memory [4]. Similarly, map-tamper attacks use `bpf_map_get_fd_by_id` to reach another program's maps, paralyzing systems like Tetragon or Datadog by deleting their tail-call maps [5].
- **Verifier and JIT bugs as kernel attack surface.** The verifier itself is complex C code (thousands of lines, growing with every helper), and has had at least 22 bugs in two years [4]; a verifier bug (e.g. CVE-2022-23222, missing pointer validation) turns unprivileged bytecode into arbitrary kernel read/write, and a JIT bug lets verified-safe code hijack kernel control flow.
- **Policy bypass.** Semantic gaps and races: a hook that fires on a decision may see different state than the decision will act on (TOCTOU between check and use); a rule written against one hook may miss the intent it was written for (the DDoS rule at XDP vs. the same intent at tc); and under load, ring-buffer events can be dropped, so user-space enforcement that depends on them can fail open (the drift between Tetragon's pre-call filters and its agent-side audit).

5.3 Mitigations

Four layers, from broadest to narrowest. Disable unprivileged eBPF (`kernel.unprivileged_bpf_disabled`, default on modern distributions) so the syscall itself requires capability. Restrict capabilities per container (deny `CAP_SYS_ADMIN`/`CAP_BPF`/`CAP_PERFMON` by default; a BPF token delegates a narrow subset through a bpffs fd) so a compromised container still cannot load. Constrain what loaded programs may do: LSM-based tools (bpfbox, BPFContain) mediate the dangerous helpers and program types themselves, which is the eBPF permission model He et al. propose [5]. Audit provenance and behavior at runtime: sign or fingerprint programs at load, and watch for the map-tamper pattern (`bpf_map_get_fd_by_id` against foreign IDs).

The prototype of Assignment \2 applies the first two: it runs with the minimum capability set (measured before and after) and on a kernel with unprivileged BPF disabled, and its threat-model audit (milestone M5) walks each attack above against the deployed configuration.

6 Limitations and open challenges

The sections above share one theme: eBPF's safety story is a verification story, and verification guarantees less than it appears to.

The verifier is part of the TCB, and a growing one. The verifier guarantees memory safety, termination and type safety for the bytecode it accepts [2], but it is ad-hoc C code that has grown with every helper and program type, has had at least 22 bugs in two years, and admits escape hatches (helpers) that bypass the very guarantees it exists to provide [4]. The community responses under way, fuzzing, verifying the verifier, or proof-carrying code, all assume the verifier stays; Jia et al. argue the deeper fix is moving extensions to a safe language (Rust) with runtime checks, eliminating the in-kernel verifier.

Helpers weaken the guarantee at the boundary. A verified program calling `bpf_probe_write_user` writes arbitrary process memory; one calling `bpf_map_get_fd_by_id` reaches another program's maps. The verifier cannot help, because the hazard is in what the helpers are **for** (tracing, inspection), not in what the bytecode does. This is why the container threat model of [5] exists at all: no kernel bug is needed, only a container with eBPF capability and a kernel that permits these helpers.

Observability and enforcement trade availability for precision. Hooks that merely observe (kprobes, tracepoints) are cheap and everywhere; hooks that enforce (XDP drop, BPF LSM verdict) stand in the data or execution path, where a false positive drops legitimate traffic or kills a legitimate process, and a bug in the program is a kernel-space outage. Tetragon splits the difference by filtering pre-call, but its audit path still crosses to user space, where events can be dropped under load.

Portability is solved for structure, not for semantics. CO-RE handles layout differences across kernels [9], but a policy still behaves differently across hook ordering changes, LSM sets and kernel versions; "compile once, run everywhere" does not free the operator from testing per deployment kernel.

The permission model lags the attack surface. Capabilities gate the loading of programs, but once loaded, tracing programs see the whole kernel; the capability split of Linux 5.6 and the BPF token are coarse answers to a problem [5] show needs per-program, per-process confinement (which is the gap bpfbox and BPFContain address at the policy layer).

7 Proposed prototype and evaluation criteria

7.1 Scope

A **continuous application-behavior profiler with a detection layer**: an eBPF-based collector that streams process- and syscall-level telemetry from two hook families, and a user-space consumer that flags the behavior patterns of Section 5 as likely malware. No in-kernel enforcement, detection only: the security property demonstrated is **detection fidelity at deployable overhead**, i.e. that a rule- and statistics-based detector over an eBPF event stream can distinguish the attacks of the threat model from clean baseline behavior with an acceptable false-positive rate and bounded per-event cost.

The two hook families answer different questions about the same process, and the prototype wants both:

- **CPU stack sampling** via a `perf_event`-driven program: periodically sample kernel and user stack traces at a fixed rate, aggregating where a process actually spends time. This is the classic continuous-profiler signal: it answers **what is this process doing** right now, independent of any syscall it makes.
- **Exec and syscall telemetry** via scheduler tracepoints (sched_switch, sched_process_exec, sched_process_fork/exit) and security-relevant tracepoints/kprobes: which binaries run, with which parentage, and which security-relevant syscalls they make. This answers **what did this process become**, and carries the behavioral signal the detector consumes: exec chains, namespace operations, unusual file access and process manipulation.

The detection layer is a user-space consumer of this stream, using standard algorithms rather than anything learned: per-process rate thresholds (events per second per binary), n-gram sequence analysis over exec and syscall ordering, and simple statistical baselines (z-scores over a clean-window profile) compared against the attack behaviors of Section 5: containerized processes attempting `bpf_probe_read_user` on foreign memory, unusual exec chains from compromised workloads, and namespace-escape attempts. Both layers together form one pipeline: profiling as the collection layer, detection as the policy applied to it, the same substrate split as Section 4′ systems.

7.2 Metrics

Table 3. Evaluation criteria for Assignment #2.

Metric	How it is measured	Target / baseline
Correctness	Attacks of Section 5 replayed against a clean baseline: detected / missed / false alarm per attack	100% of replayed attacks flagged
Throughput / event rate	Events per second sustained by the full pipeline (probe + ring buffer + consumer) before loss	Sustains peak telemetry without drops
Latency	Time from attack syscall to detector verdict	Sub-second
CPU overhead	`perf stat` and time-series comparison, idle and under benchmark load	Under 5% at 10 Hz sampling
Memory / map usage	`bpftool map` sizes: stack aggregation maps, ring-buffer occupancy	Bounded, no growth over runs
Event loss / false positives	Ring-buffer drop counter + false-positive rate over a clean-soak window	Zero drops at target rate; FPR reported, not hidden
Privileges required	Capabilities required (incl. `perf_event` access) vs. Section 5' minimum-privilege mitigation	Only `CAP_BPF` + `CAP_PERFMON`, no `CAP_SYS_ADMIN`

7.3 Environment

Isolated QEMU/KVM virtual machine(s), snapshot before each run: no public Internet targets, all attacks replayed against our own processes and containers. Kernel version pinned and recorded (a recent LTS with BTF enabled, so CO-RE relocations resolve without a patched kernel), clang + libbpf versions recorded with the artefacts. Load is generated by the benchmark suites themselves (Phoronix tests used by BPFContain's evaluation, plus a traffic generator for the network-facing probes), so overhead is measured under the same workload the baseline runs. The detector runs natively, not in a fallback mode; if the VM lacks `perf_event` access the whole exercise stops, since the privilege audit is a deliverable.

8 Implementation roadmap for Assignment #2

Table 4. Milestones, dependencies and validation. Weeks count back from the Assignment #2 deadline of 6 December 2026 (week of 30 Nov).

#	Milestone and deliverable	Depends on	Validation	Week
M1	Minimal program loads and attaches	TBD	`bpftool prog` shows the attached hook	1
M2	User-space controller / telemetry	M1	Events arrive in the ring buffer at idle and under load	1
M3	Policy and test cases	M2	Detector fires on all Section 5 replays, silent on clean soak	2
M4	Benchmark harness and baseline	M3	Metrics table targets met at 10 Hz	3–4
M5	Threat-model validation (privilege audit)	M3	Loads and runs with `CAP_BPF` + `CAP_PERFMON` only	4
M6	Reproducibility artefacts and README	M4, M5	Fresh clone of the VM reproduces one full run	5

9 Ethical and legal considerations

All experiments run in isolated virtual machines under our own control, with snapshots taken before each run; nothing touches production systems, third-party infrastructure, or the public Internet. The offensive techniques surveyed in Section 5 (cross-container reads, namespace escapes, credential manipulation) are studied as **detections**, and the ones replayed in Assignment #2 are replayed only against processes and containers we launched ourselves, in the same VM as the detector. The prototype collects telemetry from our own workloads only; it is not deployed on any system serving real users, so no user data enters the event stream and no privacy question arises beyond keeping the artefacts off shared infrastructure. The privilege-reduction results of the metrics table are the ethical payload of the exercise: if the detector needs only `CAP_BPF` and `CAP_PERFMON`, the deployment story is one of **less** privilege than the alternatives it replaces, not more.

10 Conclusion

The comparison in this report shows one substrate under four systems, and three findings fall out of it. First, the hook **is** the security decision: the same verified bytecode enforces packet-level filtering at XDP, workload-level policy at the socket layer, and reference monitoring at the BPF LSM, and the two use cases show one question (split-second triage of junk traffic vs. identity-aware control) selecting different hooks for reasons that are economic, not technical. Second, the production and research systems are the same architecture at different points of one design space, and what actually separates them is not capability but policy authorship: who defines identity and what happens when the control plane is compromised. Third, and least settled: the verifier that makes all of this safe is itself a growing, hand-written trusted computing base, one that the literature argues cannot scale as kfuncs and extensions multiply [4], and the cross-container attacks of [5] show that a shared kernel with tracing attached is a shared attack surface regardless of what the verifier proved about the programs.

The main open challenge that follows is measurable safety: neither production system publishes overhead comparable to the research benchmarks, and no system publishes detection fidelity against a named threat model. The prototype of Assignment #2 addresses exactly that gap at the smallest useful scale: a continuous profiler feeding a rule-based detector, evaluated for detection fidelity, overhead and required privilege against the threat model of Section 5. The result, if the metrics table's targets hold, is the pattern this report argues for throughout, eBPF deployed as detection with the least privilege that still answers the question, with every number published.

Bibliography

[1] The Linux Kernel Documentation, "BPF Documentation." [Online]. Available: https://docs.kernel.org/bpf/

[2] T. Høiland-Jørgensen *et al.*, "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel," in *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '18)*, ACM, 2018, pp. 54–66. doi: 10.1145/3281411.3281443.

[3] The Linux Kernel Documentation, "LSM BPF Programs." [Online]. Available: https://docs.kernel.org/bpf/prog_lsm.html

[4] J. Jia, R. Sahu, A. Oswald, D. Williams, M. V. Le, and T. Xu, "Kernel Extension Verification is Untenable," in *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*, ACM, 2023, pp. 150–157. doi: 10.1145/3593856.3595892.

[5] Y. He *et al.*, "Cross Container Attacks: The Bewildered eBPF on Clouds," in *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/he

[6] Cilium Authors, "BPF and XDP Reference Guide." [Online]. Available: https://docs.cilium.io/en/stable/reference-guides/bpf/

[7] The Linux Kernel Documentation, "eBPF Verifier." [Online]. Available: https://docs.kernel.org/bpf/verifier.html

[8] E. Gershuni *et al.*, "Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '19)*, ACM, 2019, pp. 1069–1084. doi: 10.1145/3314221.3314590.

[9] libbpf Authors, "libbpf-bootstrap: Scaffolding for BPF Application Development with libbpf and BPF CO-RE." [Online]. Available: https://github.com/libbpf/libbpf-bootstrap

[10] W. Findlay, D. Barrera, and A. Somayaji, "BPFContain: Fixing the Soft Underbelly of Container Security." [Online]. Available: https://arxiv.org/abs/2102.06972

[11] Cilium Authors, "Tetragon: eBPF-based Security Observability and Runtime Enforcement." [Online]. Available: https://tetragon.io/docs/

[12] W. Findlay, A. Somayaji, and D. Barrera, "bpfbox: Simple Precise Process Confinement with eBPF," in *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop (CCSW '20)*, ACM, 2020. doi: 10.1145/3411495.3421358.